hongzhi.zhang docs: 更新为撞库+进件接口 7bf7457 · 4 minutes ago

477 lines (391 loc) · 8.96 KB

📡 信贷流量平台 API 接口文档

一、接口概览

1.1 基础信息

项目	值
基础URL	`https://api.example.com`
数据格式	JSON
编码	UTF-8

1.2 接口列表

接口	方法	加密方式	说明
/api/v1/query/collision	POST	MD5	撞库查询
/api/v1/query/collision/batch	POST	MD5	批量撞库
/api/v1/entry/create	POST	RSA	进件提交
/api/v1/entry/batch	POST	RSA	批量进件
/api/v1/entry/status	GET	-	进件状态查询

二、撞库接口（MD5）

2.1 单条撞库

```
POST /api/v1/query/collision
```

请求头

参数	说明
Content-Type	application/json
X-App-Id	渠道标识
X-Signature	MD5签名

请求体

```
{
  "idCard": "110101199001011234",
  "phone": "13800138000",
  "name": "张三"
}
```

字段说明

字段	类型	必填	说明
idCard	string	✅	身份证号（MD5加密传输）
phone	string	✅	手机号（MD5加密传输）
name	string	✅	姓名（MD5加密传输）

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "queryId": "Q20240320001",
    "result": {
      "totalCount": 3,
      "platforms": [
        {"name": "平台A", "count": 2, "overdueCount": 0},
        {"name": "平台B", "count": 1, "overdueCount": 1}
      ],
```

```
      "riskLevel": "中风险"
    },
    "queryTime": "2024-01-15T14:30:00Z"
  }
}
```

2.2 批量撞库

```
POST /api/v1/query/collision/batch
```

请求体

```
{
  "dataList": [
    {"idCard": "xxx", "phone": "xxx", "name": "xxx"},
    {"idCard": "xxx", "phone": "xxx", "name": "xxx"}
  ]
}
```

响应

```
{
  "code": 0,
  "data": {
    "total": 10,
    "success": 10,
    "failed": 0,
    "results": [
      {"queryId": "Q001", "status": "success", "result": {...}},
      {"queryId": "Q002", "status": "success", "result": {...}}
    ]
  }
}
```

三、进件接口（RSA）

3.1 进件提交

```
POST /api/v1/entry/create
```

请求头

参数	说明
Content-Type	application/json
X-App-Id	渠道标识
X-Signature	MD5签名

请求体（RSA加密）

```
{
  "encryptedData": "BASE64编码的RSA加密数据"
}
```

加密数据结构（加密前）

```
{
  "basicInfo": {
    "name": "张三",
    "idCard": "110101199001011234",
    "phone": "13800138000",
    "gender": "男",
    "age": 35,
    "education": "本科",
    "marriage": "已婚"
  },
  "financeInfo": {
    "income": "20000",
    "socialSecurity": true,
    "housingFund": true,
    "carOwner": false
  },
  "loanInfo": {
    "amount": "50000",
    "purpose": "经营周转",
    "term": 12
  }
}
```

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "entryId": "E20240320001",
    "status": "pending",
    "createTime": "2024-01-15T14:30:00Z"
```

```
  }
}
```

3.2 批量进件

```
POST /api/v1/entry/batch
```

请求体

```
{
  "dataList": [
    {"encryptedData": "BASE64..."},
    {"encryptedData": "BASE64..."}
  ]
}
```

响应

```
{
  "code": 0,
  "data": {
    "total": 5,
    "success": 5,
    "failed": 0,
    "results": [
      {"entryId": "E001", "status": "pending"},
      {"entryId": "E002", "status": "pending"}
    ]
  }
}
```

3.3 进件状态查询

```
GET /api/v1/entry/status?entryId=E20240320001
```

响应

```
{
  "code": 0,
  "data": {
    "entryId": "E20240320001",
    "status": "approved",
```

```
    "amount": 50000,
    "term": 12,
    "updateTime": "2024-01-15T16:00:00Z"
  }
}
```

状态说明

status	说明
pending	待审核
reviewing	审核中
approved	已通过
rejected	已拒绝
disbursed	已放款

四、加密说明

4.1 MD5 撞库加密

算法

```
MD5(身份证号 + 手机号 + 姓名 + AppSecret)
```

示例（Node.js）

```
const crypto = require('crypto');

function md5Encrypt(idCard, phone, name, secret) {
  const raw = idCard + phone + name + secret;
  return crypto.createHash('md5').update(raw).hexdigest();
}

// 撞库加密
const encryptedData = md5Encrypt('110101199001011234', '13800138000', '张三', 'your_s
```

4.2 RSA 进件加密

算法

```
RSA公钥加密 + Base64编码
```

示例（Node.js）

```javascript
const crypto = require('crypto');

const PUBLIC_KEY = `-----BEGIN PUBLIC KEY-----
MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKBgQC5h...
-----END PUBLIC KEY-----`;

// RSA 加密
function rsaEncrypt(data, publicKey) {
  const encrypted = crypto.publicEncrypt(
    { key: publicKey, padding: crypto.constants.RSA_PKCS1_OAEP_PADDING },
    Buffer.from(JSON.stringify(data))
  );
  return encrypted.toString('base64');
}

// 进件加密
const rawData = {
  basicInfo: { name: '张三', idCard: '110101199001011234', phone: '13800138000' },
  financeInfo: { income: '20000' },
  loanInfo: { amount: '50000', term: 12 }
};

const encryptedData = rsaEncrypt(rawData, PUBLIC_KEY);
```

示例（Python）

```python
import base64
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

PUBLIC_KEY = """-----BEGIN PUBLIC KEY-----
MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKBgQC5h...
-----END PUBLIC KEY-----"""

def rsa_encrypt(data, public_key):
    key = RSA.import_key(public_key)
    cipher = PKCS1_OAEP.new(key)
    encrypted = cipher.encrypt(json.dumps(data).encode())
    return base64.b64encode(encrypted).decode()

# 使用
raw_data = {
    "basicInfo": {"name": "张三", "idCard": "110101199001011234"},
    "loanInfo": {"amount": "50000"}
```

```
}
encrypted = rsa_encrypt(raw_data, PUBLIC_KEY)
```

五、签名算法

5.1 请求签名

```
Signature = MD5(请求体JSON + AppSecret)
```

示例

```
function sign(body, secret) {
  return crypto.createHash('md5')
    .update(JSON.stringify(body) + secret).digest('hex');
}
```

六、错误码

code	message	说明
0	success	成功
1001	missing parameter	缺少参数
1002	invalid format	格式错误
1003	signature error	签名错误
1004	channel disabled	渠道禁用
1005	rate limit	频率限制
2001	encryption error	加密失败
2002	decryption error	解密失败
3001	query failed	查询失败
3002	entry failed	进件失败

七、Demo 代码

7.1 撞库查询（Node.js）

```
const axios = require('axios');
const crypto = require('crypto');

const CONFIG = {
  API_URL: 'https://api.example.com',
  APP_ID: 'your_app_id',
  APP_SECRET: 'your_app_secret'
};

function md5(str) {
  return crypto.createHash('md5').update(str).hexdigest();
}

function sign(body) {
  return md5(JSON.stringify(body) + CONFIG.APP_SECRET);
}

async function queryCollision(idCard, phone, name) {
  const body = { idCard, phone, name };
  const response = await axios.post(
    `${CONFIG.API_URL}/api/v1/query/collision`,
    body,
    {
      headers: {
        'Content-Type': 'application/json',
        'X-App-Id': CONFIG.APP_ID,
        'X-Signature': sign(body)
      }
    }
  );
  return response.data;
}

// 使用
queryCollision('110101199001011234', '13800138000', '张三')
  .then(console.log);
```

7.2 进件提交（Node.js）

```
const axios = require('axios');
const crypto = require('crypto');

const CONFIG = {
  API_URL: 'https://api.example.com',
  APP_ID: 'your_app_id',
```

```
  APP_SECRET: 'your_app_secret',
  RSA_PUBLIC_KEY: `-----BEGIN PUBLIC KEY-----\n...`
};

function sign(body) {
  return crypto.createHash('md5')
    .update(JSON.stringify(body) + CONFIG.APP_SECRET).hexdigest();
}

function rsaEncrypt(data) {
  const encrypted = crypto.publicEncrypt(
    { key: CONFIG.RSA_PUBLIC_KEY, padding: crypto.constants.RSA_PKCS1_OAEP_PADDING }
    Buffer.from(JSON.stringify(data))
  );
  return encrypted.toString('base64');
}

async function createEntry(data) {
  const encryptedData = rsaEncrypt(data);
  const response = await axios.post(
    `${CONFIG.API_URL}/api/v1/entry/create`,
    { encryptedData },
    {
      headers: {
        'Content-Type': 'application/json',
        'X-App-Id': CONFIG.APP_ID,
        'X-Signature': sign({ encryptedData })
      }
    }
  );
  return response.data;
}

// 使用
const entryData = {
  basicInfo: { name: '张三', idCard: '110101199001011234', phone: '13800138000' },
  financeInfo: { income: '20000' },
  loanInfo: { amount: '50000', term: 12 }
};

createEntry(entryData).then(console.log);
```

八、测试

测试账号

AppId	AppSecret
test_001	test_secret_001

测试命令

```
# 撞库测试
curl -X POST https://api.example.com/api/v1/query/collision \
  -H "Content-Type: application/json" \
  -H "X-App-Id: test_001" \
  -H "X-Signature: $(echo -n '{"idCard":"xxx"}test_secret_001' | md5)" \
  -d '{"idCard":"xxx","phone":"xxx","name":"xxx"}'
```

文档版本: v1.0
更新时间: 2024-01-15